

Algorithmen und Datenstrukturen

Übung 3 – Objektorientierte Programmierung

Aufgabe 1: Theorie

a) Beantworten Sie zu dem nachstehenden Codestück die folgenden Fragen:

- Was ist das Problem an dem Codeausschnitt?
- Wie äußert sich das Problem allgemein und vermutlich im Speziellen?
- Wieso können der Compiler und der Linker das Problem nicht finden?

Hinweis: Lassen Sie sich gegebenenfalls den Speicherinhalt anzeigen.

```
1  // numerics.hpp
2  #if !defined(INCLUDED__BLATT5__NUMERICS_HPP)
3  #define INCLUDED__BLATT5__NUMERICS_HPP
4      #if !defined(PRECISION)
5          // No explicit precision selected.
6          #define PRECISION double
7      #endif

8      struct vector { PRECISION x, y, z; };
9      vector my_vector();

10 #endif /* !defined(INCLUDED__BLATT5__NUMERICS_HPP) */

11 // numerics.cpp
12 #include "numerics.hpp"
13 #include <iostream>

14 vector my_vector()
15 {
16     vector n;
17     n.x = n.y = n.z = 1234;
18     std::cout << n.x << " " << n.y << " " << n.z;
19     return n;
20 }

21 // main.cpp
22 #define PRECISION float
23 #include <iostream>
24 #include "numerics.hpp"

25 int main()
26 {
27     vector n(my_vector());
28     std::cout << n.x << " " << n.y << " " << n.z;
29 }
```

b) Was für Probleme ergeben sich mit folgendem Codeausschnitt und wie sind die Probleme zu beheben?

```
1  #include <string>
2  class resource_id
3  {
4      public:
5          const std::string full_path;
6
7          resource_id(const std::string& base_path,
8                    const std::string& file_title,
9                    const std::string& extension)
10             :
11             file_name_(file_title + "." + extension),
12             full_path(base_path_ + "/" + file_name_) { }
13
14     private:
15         const std::string base_path_;
16         const std::string file_name_;
17 };
```

c) Diskutieren Sie in den folgenden, unabhängigen Fällen, ob es zu einem Compiler- oder Linker-Fehler, zu undefiniertem oder zu korrektem Verhalten kommt:

- Ein Header definiert eine freie Funktion `int a(float x, float y)`. Der Header wird von mehreren `.cpp`-Dateien verwendet.
- Ein Header definiert eine freie Funktion `int b()`. Eine `.cpp`-Datei bindet den Header ein und definiert anschließend selbst eine freie Funktion `int b()`.
- Ein Header deklariert eine freie Funktion `int c(float x)`. Zu der Funktion gibt es in zwei verschiedenen `.cpp`-Dateien jeweils eine identische Definition.
- Ein Header `d.hpp` deklariert eine freie Funktion `int d(float x)`. Zu der Funktion gibt es in `d.cpp` eine Definition, aber `d.cpp` inkludiert `d.hpp` auf keinem Weg.
- Ein Header deklariert und definiert eine Klasse `E`. Für die Methoden der Klasse gibt es in zwei verschiedenen `.cpp`-Dateien identische Definitionen.
- Ein Header deklariert und definiert eine Klasse `F`, alle Methoden der Klasse sind `inline`-definiert. Der Header wird in mehreren `.cpp`-Dateien eingebunden.
- Mehrere `.cpp`-Dateien definieren jeweils unterschiedliche Klassen `H` und hängen nach Bedarf Instanzen der Klasse `H` an einen globalen `std::unique_ptr<H>` an.

d) Angenommen, man definiert Funktionen grundsätzlich in den Klassen statt außerhalb davon (egal ob im Header oder in der Source).

- Wie könnte man naiv argumentieren, dass dieses Vorgehen die Abarbeitungsgeschwindigkeit des Programms erhöht? Geben Sie zur Begründung an, welche Rechenoperationen wegfallen.
- Recherchieren Sie selbst, wie die Branch Prediction in einer modernen CPU grundsätzlich funktioniert. Erklären Sie auf Grundlage Ihrer Erkenntnisse, wieso ein regelmäßiges Vorgehen wie oben das Programm in vielen praktischen Fällen langsamer macht.

- Recherchieren Sie, was Whole Program Optimization ist. Welchen Einfluss hat diese Art der Optimierung auf das obige Vorgehen?

e) Angenommen, eine Datei `robot.cpp` umfasst unter anderem die folgende Definition einer `battery`-Klasse

```
class battery { public: battery() { std::cout << "robot on"; } };
```

während eine Datei `sensor.cpp` die folgende, abweichende Definition

```
class battery { public: battery() { std::cout << "sensor on"; } };
```

enthält. Instanzen der Klasse `battery` werden **an keiner Stelle** zwischen beiden Varianten ausgetauscht (d. h. Instanzen von `battery` werden je nur lokal innerhalb der `.cpp`-Dateien genutzt und nicht hin- und hergegeben).

Das Programm kompiliert und linkt scheinbar korrekt, erzeugt aber undefiniertes Verhalten. Warum passiert das und wie äußert sich das undefinierte Verhalten in diesem Fall vermutlich?

Hinweise:

- Das undefinierte Verhalten wird sich nur unter ziemlich speziellen Bedingungen (hier: Compiler- und Linker-Einstellungen) zeigen und nur, wenn der Compiler und der Linker die nötigen Annahmen treffen.
- Welche Optimierung könnte der Linker durchführen, unter der Annahme, dass beide Definitionen von `battery` identisch sind? Berücksichtigen Sie dabei Ihre Antwort zu Teilaufgabe 1d), Punkt 2.

Aufgabe 2: Praxis

a) In einer GUI kann ein Fensterelement der Klasse `window` auf Basis verschiedener Ausgangsdaten erzeugt werden. Geben Sie geeignete Deklarationen für (sinnvoll überladene und benannte) Methoden an, die das Hauptfenster aus den gegebenen Daten erzeugen. Beachten Sie, dass Konstruktoren auch Methoden sind.

- ein leeres Hauptfenster mit Standardeigenschaften
- ein Hauptfenster mit einer bestimmten Größe in X- und Y-Richtung
- ein Hauptfenster mit einer bestimmten Überschrift
- ein Fenster, das innerhalb eines Elternfensters angezeigt wird
- ein Hauptfenster, das beliebig viele gegebene Kindfenster besitzt
- ein Fenster, dessen Eigenschaften und Kindfenster aus einer Ressourcendatei mit angegebenem Pfad geladen werden

b) Erstellen Sie eine Klasse `int_smart_pointer` mit folgenden Eigenschaften:

- Jeder `int_smart_pointer` hat immer einen zugeordneten `int` als Allokation auf dem Heap liegen.
- Der `int_smart_pointer` merkt sich intern, wo der `int` auf dem Heap liegt.
- Von außerhalb des `int_smart_pointer` lässt sich der `int` mit Methoden `set` und `get` auslesen und verändern, aber weder löschen noch neu anlegen.
- Man kann beim Anlegen einer Instanz von `int_smart_pointer` den Anfangswert des `ints` vorgeben, sonst wird als Wert 0 genommen.
- Wenn man einen `int_smart_pointer` kopiert, ist das Kopieren *tiefe*: der neue `int_smart_pointer` hat automatisch eine neue Heap-Allokation, der darin gespeicherte `int` bekommt den selben Wert wie der Quell-`int`.
- Wenn ein `int_smart_pointer` gelöscht wird, gibt es kein Speicherleck.
- Alle Operationen auf `int_smart_pointer` schreiben auf die Kommandozeile, dass die jeweilige Operation gerade läuft (d. h. die Methoden geben den jeweils eigenen Namen aus).

Prüfen Sie an folgenden Codezeilen, ob `int_smart_pointer` korrekt funktioniert:

```
int_smart_pointer a;
std::cout << a.get() << std::endl;           // Ausgabe 0

a.set(5);
std::cout << a.get() << std::endl;           // Ausgabe 5

int_smart_pointer b(10);
std::cout << b.get() << std::endl;           // Ausgabe 10

int_smart_pointer c(b);
std::cout << c.get() << std::endl;           // Ausgabe 10

c.set(20);
std::cout << b.get() << std::endl;           // Ausgabe 10
std::cout << c.get() << std::endl;           // Ausgabe 20
```

c) Passen Sie den Code der vorigen Aufgabe an: Wenn man den `int_smart_pointer` kopiert, soll die Kopie *flach* erfolgen, also den selben (d. h. identischen) `int` wie das Original verwenden. Was geht jetzt schief? Prüfen Sie ggf. mit den obigen Codezeilen, beachten Sie aber, dass sich der Fehler nicht unbedingt äußern muss.

d) Schreiben Sie eine Klasse `rabbit` für niedliche Häschen. Ein `rabbit` hat folgende Eigenschaften:

- einen Namen, gespeichert als `std::string`
- eine Ganzzahl für die Anzahl an Ohren, die der Hase hat
- ein Alter in Wochen als Ganzzahl
- ein Gewicht in Gramm als Gleitkommazahl
- einen Wahrheitswert, ob der Hase aktuell essbar ist

- Ein `rabbit` lässt sich mit gegebenem Namen und gegebener Ohrenzahl anlegen. Anfänglich wiegt der Hase 100 Gramm und er ist 0 Wochen alt.
- Es gibt eine Funktion `altern()`, die das Alter des Hasen um 1 Woche erhöht.
- Es gibt eine Funktion `füttern()`, welche das Gewicht des Hasen um eine zufällige Zahl zwischen 10 und 200 erhöht.
- Der Wahrheitswert `essbar` ist wahr, wenn das Alter zwischen 9 und 12 Wochen und das Gewicht zwischen 1500 und 2000 Gramm liegt. Der Wert soll ständig mit dem Alter und dem Gewicht des Hasen zusammenpassen. Man kann den Wert von außen abfragen, aber nicht verändern.

Schreiben Sie eine `main`-Funktion, um Ihre Hasenklasse zu testen. Legen Sie ein paar Hasen mit Beispieldaten an, lassen Sie diese altern, und prüfen Sie, ob die Hasen korrekt essbar werden.

- e) Erweitern Sie zunächst die `rabbit`-Klasse so, dass Hasen sich an der Hand halten können (vgl. ausgeschnittene Häskenkette): Zum Händchenhalten hat jeder Hase eine rechte und eine linke Hand, wobei an der rechten wie an der linken Hand jeweils ein anderer Hase oder kein Hase hängen kann.

Schreiben Sie weiterhin eine Klasse `stall`, für einen Ort an dem Ihre Hasen glücklich leben können:

- Im Stall halten sich die Hasen in einer durchgehenden Kette an der Hand. Es gibt also einen ersten Hasen, der keinen Hasen an seiner linken Hand hat, und einen letzten Hasen, der keinen Hasen an seiner rechten Hand hat. Alle anderen Hasen haben einen Partner an der linken und der rechten Hand.
- Der Stall selbst merkt sich nur den ersten Hasen in der Kette.
- Der Stall hat eine Funktion `hinzufügen`, die einen neuen Hasen anlegt und an die bestehende Häskenkette hinten anhängt. Das heißt also, dass der (vormals) letzte Hase einen neuen Partner für seine rechte Hand bekommt. Der neu angelegte Hase hat eine leere rechte Hand.
- Der Stall hat eine Funktion `züchten`, die alle Hasen altern lässt und füttert.
- Der Stall hat eine Funktion `ernten`. Diese Funktion geht alle Hasen durch. Wenn ein Hase essbar ist, wird der Hase aus dem Stall entfernt. Wenn ein Hase älter als 50 Wochen ist, wird der Hase ebenfalls entfernt.
- Es gibt eine Funktion `inventarisieren`. Diese Funktion geht alle Hasen im Stall durch und gibt deren Daten auf der Kommandozeile aus.

Testen Sie Ihren `stall` wie folgt: Fügen Sie ein paar Hasen in den Stall ein. Züchten und ernten Sie die Hasen abwechselnd, bis der Stall leer ist. Geben Sie nach jedem Schritt per prüfen den Zustand des Stalls aus.